

Finance Research Agent — Updated Master Implementation Plan

Expanded architecture: React + FastAPI + Flue + PostgreSQL, while preserving the original assignment scope

1. Project Objective

- Build a complete Finance Research Agent using the Flue Agent Framework that accepts Indian stock symbols and produces structured, evidence-grounded research notes.
- Support single-stock research and two-stock comparison.
- Extend the assignment into a deployable application with a React frontend, FastAPI backend, and PostgreSQL persistence layer while preserving the original agent/tool architecture.
- Do not implement real trading, order placement, broker execution, or portfolio execution.

2. Proposed Technology Stack

- Frontend: React + Vite + TypeScript for stock search, research reports, confidence visualization, bullish/risk evidence, and comparison.
- Backend: FastAPI + Python as the API/application boundary.
- Agent layer: Flue Agent Framework for agent orchestration and tool-using/ReAct-style execution.
- Finance engine: deterministic Python logic for returns, SMA20/SMA50, volatility, momentum, 52-week positioning, signal score, recommendation, and confidence.
- Market data: provider abstraction with live provider and mock provider.
- Database: PostgreSQL for stock metadata, market snapshots, analysis runs, indicators, signals, and generated reports.
- ORM/data access: SQLAlchemy 2.x.
- Validation/schema: Pydantic / Pydantic Settings.
- Testing: pytest plus FastAPI/httpx testing where appropriate.
- Configuration: environment variables and .env.example; secrets never committed.
- Containerization: Docker and Docker Compose for reproducible backend/database development.
- Version control: Git and GitHub.

3. Target Architecture

- React UI calls FastAPI endpoints.
- FastAPI validates requests, exposes application APIs, invokes the Flue agent workflow, and coordinates persistence.
- Flue remains the agent orchestration layer and brain for interpretation/reasoning.
- Flue tools invoke deterministic finance and market-data functions.
- PostgreSQL stores durable application/research history; it is not used to perform finance calculations.
- Agent-generated thesis and reports must be grounded in deterministic tool outputs.
- Frontend is presentation only; business rules remain in backend/domain layers.

4. Core Agent Workflow

- Single-stock flow: input → validation → fetch_market_data → calculate_indicators → analyze_bullish_factors → analyze_risk_factors → generate_investment_thesis → calculate_confidence → produce_final_report → validation → persistence → API response.
- Comparison flow: validate two symbols → independently execute market data/indicators/signals/confidence → compare structured evidence → generate comparison reasoning/report → persist → return.
- Use the actual Flue APIs discovered during Phase 0; do not invent a parallel agent framework.

5. Market Data Layer

- Required data: current price, previous close, daily percentage change, 52-week high/low, volume, and basic recent price history.
- Create a provider-independent MarketDataProvider abstraction with live and mock implementations.
- Use a mock universe of 5–10 Indian stocks when live data is unavailable.
- Persist selected market snapshots to PostgreSQL with timestamps and data-source metadata.

6. Deterministic Finance Engine

- Daily return percentage.
- Distance from 52-week high and low.
- SMA20 and SMA50 when sufficient history exists.
- Momentum using price relative to SMA20/SMA50 and recent direction.
- Volatility using recent daily returns.
- All core financial mathematics must be deterministic and independently testable.

7. Signals, Recommendation & Confidence

- Bullish evidence: positive daily return, price above SMA20/SMA50, positive momentum, strong 52-week positioning.
- Risk evidence: negative daily return, price below SMA20/SMA50, weak momentum, high volatility, and large distance from the 52-week high.
- Recommendation categories: BUY, HOLD, WATCHLIST, AVOID.
- Enhanced confidence weighting: signal strength 30%, bullish/bearish consistency 25%, indicator agreement 20%, data completeness 15%, decision stability 10%.
- Confidence is evidence strength, not a probability that a stock will rise.

8. Agent Hooks & Validation

- Investigate actual Flue hook support before implementation.
- Where supported, use hooks for input validation, tool lifecycle observability, reasoning/evidence consistency, deterministic confidence attachment, and final report validation.
- Never fake unsupported hooks. Document framework limitations if a required lifecycle hook is unavailable.

9. Backend API

- POST /api/research — single-stock research.
- POST /api/compare — two-stock comparison.
- GET /api/health — service health.
- Backend should return structured JSON suitable for the React frontend.
- API validation and error handling must be separate from agent reasoning.

10. PostgreSQL Persistence

- stocks: symbol, company metadata, exchange information.
- market_snapshots: price, previous close, daily change, 52-week range, volume, timestamp, source.
- analysis_runs: analysis ID, symbol(s), timestamp, recommendation, confidence, status.
- indicators: analysis ID, daily return, SMA20, SMA50, momentum, volatility, 52-week distances.
- signals: analysis ID, type, evidence, strength.
- reports: analysis ID, thesis, reasoning, structured report payload.
- Use SQLAlchemy and migrations as appropriate; database must not become a hidden dependency for deterministic calculations.

11. Frontend

- React + Vite + TypeScript.
- Stock analysis input and loading/error states.
- Structured research report view.
- Confidence score visualization.
- Bullish/risk factor display.
- Indicator display.
- Recommendation display.
- Two-stock comparison view.
- Keep frontend free of finance calculations; consume backend results.

12. Testing

- Unit tests for market-data normalization, indicators, signals, scoring, confidence, and report validation.
- Tool tests for each Flue tool.
- Agent/workflow tests for correct orchestration.
- API tests for research, comparison, health, validation, and failures.
- Database integration tests where persistence behavior matters.
- End-to-end test from API request through agent/tools to persisted result and response.

13. Documentation & Submission

- README must document architecture, Flue integration, tools, hooks, frontend, backend, PostgreSQL, indicators, signal logic, confidence, APIs, setup, configuration, testing, sample reports, live/mock data, design decisions, limitations, and disclaimer.
- Provide at least three stock reports and one comparison report.
- Document which components are live and which are mocked.
- Use a separate feature branch and maintain focused commits.

Master Plan → Phase Plan Mapping

Area	Updated Master Plan location
Original assignment	Preserved: agent workflow, market data, indicators, reasoning, report, comparison, testing, no trading
Flue	Core agent/orchestration layer; actual API discovered in Phase 0
Frontend	New React + Vite + TypeScript presentation layer
Backend	New FastAPI + Python API/application layer
Database	New PostgreSQL persistence layer with SQLAlchemy
Finance engine	Deterministic Python domain layer
Hooks	Conditional Flue hook integration based on actual framework support
Testing	Unit + tool + agent/workflow + API + database + E2E
Deployment/dev	Docker + Docker Compose for reproducible local services